

Contents

1	Experiment : Characteristics of LED and Semiconductor LASER	3
1.1	Objectives	3
1.2	Theory and Relevant Equations	3
1.3	Simulation Parameters	3
1.4	Procedure	3
1.5	Results and Generated Figures	4
1.6	Parameter Comparisons and Model Limitations	4
1.7	Complete MATLAB Code	4
2	Experiment : Optical Modulation Using MZM and EAM	9
2.1	Objectives	9
2.2	Theory and Relevant Equations	9
2.3	Simulation Parameters	9
2.4	Procedure	10
2.5	Results and Generated Figures	10
2.6	Parameter Comparisons and Model Limitations	10
2.7	Complete MATLAB Code	11
3	Experiment : Noise, SNR, Q-Factor, and BER of PIN and APD Receivers	16
3.1	Objectives	16
3.2	Theory and Relevant Equations	16
3.3	Simulation Parameters	16
3.4	Procedure	17
3.5	Results and Generated Figures	17
3.6	Parameter Comparisons and Model Limitations	17
3.7	Complete MATLAB Code	17
4	Experiment : Power Budget and Rise-Time Budget of a Fiber-Optic Link	23
4.1	Objectives	23
4.2	Theory and Relevant Equations	23
4.3	Simulation Parameters	23
4.4	Procedure	23
4.5	Results and Generated Figures	24
4.6	Parameter Comparisons and Model Limitations	24
4.7	Complete MATLAB Code	24
5	Experiment : Line Coding and Eye-Pattern Analysis	28
5.1	Objectives	28
5.2	Theory and Relevant Equations	28
5.3	Simulation Parameters	28
5.4	Procedure	28
5.5	Results and Generated Figures	28
5.6	Parameter Comparisons and Model Limitations	29
5.7	Complete MATLAB Code	30
6	Experiment : WDM Spectrum, Fiber Attenuation, and OTDR Simulation	34
6.1	Objectives	34
6.2	Theory and Relevant Equations	34
6.3	Simulation Parameters	34
6.4	Procedure	35

6.5	Results and Generated Figures	35
6.6	Parameter Comparisons and Model Limitations	35
6.7	Complete MATLAB Code	36

1 Experiment : Characteristics of LED and Semiconductor LASER

1.1 Objectives

- Compare the optical output power characteristics of an LED and a semiconductor LASER under varying injection currents.
- Understand and determine the LASER threshold current (I_{th}) and external differential quantum efficiency.
- Analyze and compare the electrical-to-optical wall-plug efficiencies of both light sources.
- Evaluate and compare the small-signal modulation responses and frequency bandwidths of LED and LASER devices.

1.2 Theory and Relevant Equations

An LED primarily operates through spontaneous emission, generating incoherent light whose output power increases approximately linearly with injection current. In contrast, a semiconductor LASER utilizes carrier injection into an active region, stimulated emission, and optical feedback from a Fabry-Perot cavity to produce coherent laser light above a specific threshold current.

The simplified LED optical output power model is expressed as:

$$P_{LED} = \eta_{ext} \frac{h\nu}{q} I \quad (1)$$

For a semiconductor LASER, the stimulated optical output power above threshold is modeled as:

$$P_{LASER} = \eta_d \frac{h\nu}{q} \max(I - I_{th}, 0) \quad (2)$$

where η_{ext} is the LED external quantum efficiency, η_d is the LASER external differential quantum efficiency, h is Planck's constant, ν is the optical frequency (c/λ), q is the electron charge, and I_{th} is the threshold current.

The electrical-to-optical wall-plug efficiency (η_{WP}) is defined as:

$$\eta_{WP} = \frac{P_{optical}}{VI} \quad (3)$$

The normalized small-signal frequency modulation responses are governed by first-order low-pass and second-order resonance dynamics respectively:

$$H_{LED}(f) = \frac{1}{1 + jf/f_c} \quad (4)$$

$$H_{LASER}(f) = \frac{f_r^2}{f_r^2 - f^2 + j2\zeta f_r f} \quad (5)$$

where f_c is the LED cutoff frequency, f_r is the LASER relaxation-resonance frequency, and ζ is the damping ratio.

1.3 Simulation Parameters

1.4 Procedure

1. Initialize fundamental physical constants (h, c, q) and set the optical operating wavelength to 850 nm.

Parameter	Baseline value
Planck's constant (h)	$6.626 \times 10^{-34} \text{ J} \cdot \text{s}$
Speed of light (c)	$3 \times 10^8 \text{ m/s}$
Electron charge (q)	$1.602 \times 10^{-19} \text{ C}$
Optical wavelength (λ)	850 nm
Injection current range (I)	0.1 mA to 100 mA (1000 points)
LED optical conversion factor (η_{LED})	0.03
Laser optical conversion factor (η_{Laser})	0.40
Laser threshold current (I_{th})	25 mA
LED / Laser forward voltages	1.8 V / 2.0 V
LED cutoff frequency (f_c)	50 MHz
Laser resonance frequency (f_r)	3 GHz ($\zeta = 0.45$)

2. Create an injection current vector spanning from 0.1 mA to 100 mA with 1000 sample points, avoiding exact zero current to prevent division-by-zero errors in efficiency calculations.
3. Compute the optical output power for the LED and semiconductor LASER using linear and threshold-limited equations.
4. Evaluate wall-plug efficiency across the current sweep using device operating voltages.
5. Construct a logarithmic frequency vector from 1 MHz to 31.6 GHz to compute the normalized small-signal modulation responses (H_{LED} and H_{LASER}).
6. Plot power-current ($P - I$) characteristics, wall-plug efficiency, and modulation frequency response curves.

1.5 Results and Generated Figures

Physical Interpretation: The LED optical power exhibits a direct linear increase with injection current from the origin. Conversely, the LASER optical power remains zero below the threshold current ($I_{th} = 25 \text{ mA}$) and rises sharply with a much higher slope efficiency above threshold. The LED demonstrates a standard first-order low-pass frequency response, while the LASER features a distinct resonant peak near its relaxation frequency ($f_r = 3 \text{ GHz}$).

1.6 Parameter Comparisons and Model Limitations

Parameter Changes: Varying I_{th} to 15, 25, and 35 mA shifts the onset of stimulated emission on the P-I curves. Changing the damping factor ζ directly controls the peaking behavior of the LASER resonance curve.

Model Limitations: Below-threshold spontaneous emission and thermal rollover effects at high operating currents are excluded from this simplified model.

1.7 Complete MATLAB Code

```
clear;
clc;
close all;

%% =====
% OPTICAL SOURCES: LED vs LASER
% Power-Current, Efficiency and Modulation Response
```

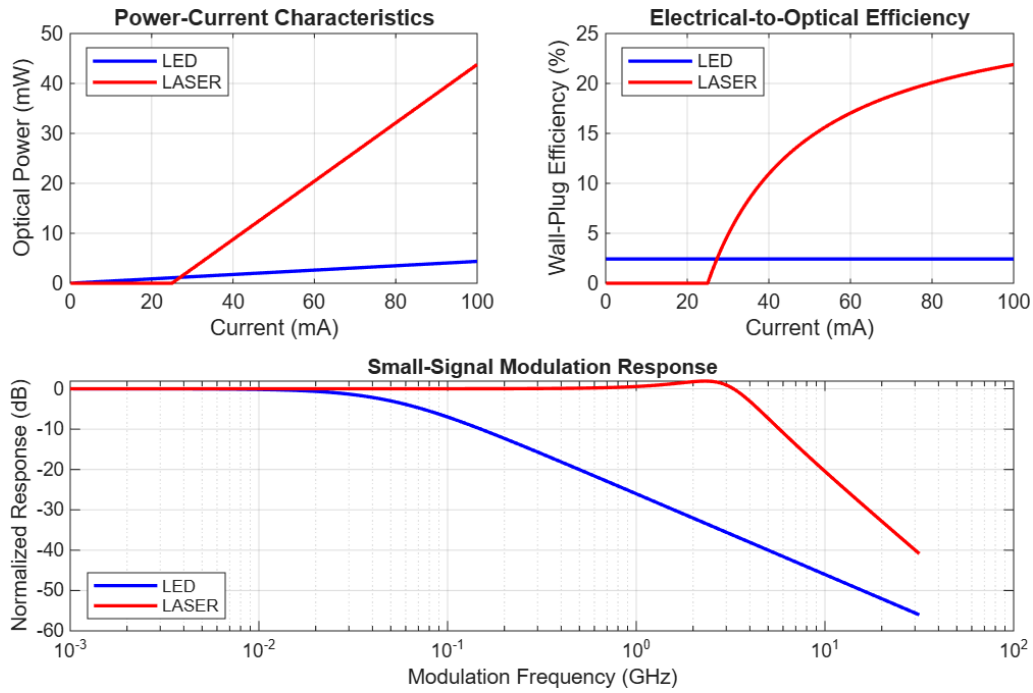


Figure 1: Power-Current Characteristics, Wall-Plug Efficiency, and Small-Signal Modulation Response for LED and LASER.

```
% =====

%% Physical Constants

h = 6.626e-34;          % Planck constant (J.s)
c = 3e8;                % Speed of light (m/s)
q = 1.602e-19;          % Electron charge (C)

lambda = 850e-9;        % Wavelength (m)

% Optical frequency
nu = c/lambda;

%% =====
% INJECTION CURRENT
% =====

% Avoid I = 0 in efficiency calculation
I = linspace(0.1e-3,100e-3,1000);

%% =====
% DEVICE PARAMETERS
% =====

etaLED = 0.03;           % LED optical conversion factor
etaLaser = 0.40;         % Laser optical conversion factor
Ith = 25e-3;             % Laser threshold current (A)
```

```

VLED = 1.8;                      % LED forward voltage (V)

VLaser = 2.0;                    % Laser voltage (V)

%% =====
% OPTICAL OUTPUT POWER
% =====

% LED optical output power
PLED = etaLED * (h*nu/q) .* I;

% Laser optical output power
% Laser produces significant output above threshold current
PLaser = etaLaser * (h*nu/q) .* max(I-Ith,0);

%% =====
% WALL-PLUG EFFICIENCY
% =====

effLED = 100 * PLED ./ (VLED*I);

effLaser = 100 * PLaser ./ (VLaser*I);

%% =====
% SMALL-SIGNAL MODULATION RESPONSE
% =====

% Frequency range
f = logspace(6,10.5,1500);

% LED bandwidth
fc = 50e6;

% Laser relaxation frequency
fr = 3e9;

% Damping factor
zeta = 0.45;

%% LED response

HLED = 1 ./ (1 + 1j*f/fc);

%% LASER response

HLaser = ...
    fr^2 ./ ...
    (fr^2 - f.^2 + 1j*2*zeta*fr.*f);

%% =====
% PLOTS
% =====

```

```

figure('Color','w');

%% 1. Power-Current Characteristics

subplot(2,2,1);

plot(I*1e3,...
      PLED*1e3,...
      'b','LineWidth',1.6);

hold on;

plot(I*1e3,...
      PLaser*1e3,...
      'r','LineWidth',1.6);

xlabel('Current (mA)');
ylabel('Optical Power (mW)');

legend('LED','LASER',...
       'Location','northwest');

title('Power-Current Characteristics');

grid on;

%% 2. Electrical-to-Optical Efficiency

subplot(2,2,2);

plot(I*1e3,...
      effLED,...
      'b','LineWidth',1.6);

hold on;

plot(I*1e3,...
      effLaser,...
      'r','LineWidth',1.6);

xlabel('Current (mA)');
ylabel('Wall-Plug Efficiency (%)');

legend('LED','LASER',...
       'Location','northwest');

title('Electrical-to-Optical Efficiency');

grid on;

%% 3. Small-Signal Modulation Response

subplot(2,2,[3 4]);

```

```

semilogx(f/1e9,...
    20*log10(abs(HLED)),...
    'b','LineWidth',1.6);

hold on;

semilogx(f/1e9,...
    20*log10(abs(HLaser)),...
    'r','LineWidth',1.6);

xlabel('Modulation Frequency (GHz)');
ylabel('Normalized Response (dB)');

legend('LED','LASER',...
    'Location','southwest');

title('Small-Signal Modulation Response');

grid on;

%% =====
% DISPLAY RESULTS
% =====

fprintf('\n');
fprintf('===== \n');
fprintf('LED vs LASER RESULTS \n');
fprintf('===== \n');

fprintf('Wavelength                = %.0f nm \n',...
    lambda*1e9);

fprintf('Optical frequency            = %.3e Hz \n',...
    nu);

fprintf('\n');

fprintf('LASER threshold current      = %.1f mA \n',...
    Ith*1e3);

fprintf('LASER slope efficiency       = %.3f W/A \n',...
    etaLaser*h*nu/q);

fprintf('LED conversion factor        = %.3f \n',...
    etaLED);

fprintf('LASER conversion factor      = %.3f \n',...
    etaLaser);

fprintf('\nSimulation completed successfully. \n');

```


2 Experiment : Optical Modulation Using MZM and EAM

2.1 Objectives

- Observe and analyze the static transfer characteristic curve of a Mach-Zehnder Modulator (MZM).
- Understand voltage-dependent optical absorption mechanisms in an Electro-Absorption Modulator (EAM).
- Compare the time-domain optical outputs and calculate extinction ratios for both external modulators.

2.2 Theory and Relevant Equations

A Mach-Zehnder Modulator (MZM) splits continuous-wave input light into two optical waveguide arms. By applying a modulating voltage, the relative optical phase between the arms is varied, producing constructive or destructive interference at the output coupler to control optical intensity. The ideal MZM transfer function is modeled as:

$$P_{out} = P_{in} \cos^2 \left(\frac{\pi V}{2V_\pi} \right) \quad (6)$$

where V is the effective modulation voltage (including bias) and V_π is the half-wave switching voltage.

An Electro-Absorption Modulator (EAM) modulates light intensity by altering semiconductor bandgap absorption through an applied reverse-bias voltage (V_R). Using Beer-Lambert's model:

$$P_{out} = P_{in} e^{-\alpha(V_R)L} \quad (7)$$

where $\alpha(V_R) = \alpha_0 + k_\alpha V_R$, α_0 is baseline absorption, k_α is the electro-absorption coefficient, and L is active modulator length. The extinction ratio (ER) in dB is defined as:

$$ER = 10 \log_{10} \left(\frac{P_1}{P_0} \right) \quad (8)$$

2.3 Simulation Parameters

Parameter	Baseline value
Input continuous-wave power (P_{in})	10 mW
MZM half-wave voltage (V_π)	4 V
MZM voltage sweep range	0 to 8 V (1000 points)
EAM reverse-bias sweep (V_R)	0 to 4 V (1000 points)
EAM active length (L)	200 μm
EAM baseline absorption (α_0)	1000 m^{-1}
EAM coefficient (k_α)	4000 $(\text{m} \cdot \text{V})^{-1}$
Input bit sequence	[1 0 1 1 0 0 1 0 1 0]
Bit rate (R_b) / Samples per bit (sps)	1 Gb/s / 100

2.4 Procedure

1. Set up input CW power ($P_{in} = 10$ mW) and MZM half-wave voltage ($V_\pi = 4$ V).
2. Compute MZM static optical transmission across a voltage vector from 0 to $2V_\pi$.
3. Configure EAM parameters (L, α_0, k_α) and evaluate static exponential transmission across a reverse-bias voltage vector from 0 to 4 V.
4. Generate a 10-bit unipolar NRZ digital data sequence at 1 Gb/s with oversampling ('sps = 100').
5. Map logical bits to respective electrical drive voltages for MZM and EAM, simulate time-domain modulated optical outputs, and calculate extinction ratios.

2.5 Results and Generated Figures

Physical Interpretation: The MZM transfer characteristic follows a periodic cosine-squared interference pattern, whereas the EAM exhibits a monotonically decreasing exponential absorption curve under increasing reverse bias. The selected drive conditions yield high extinction ratios for both modulators.

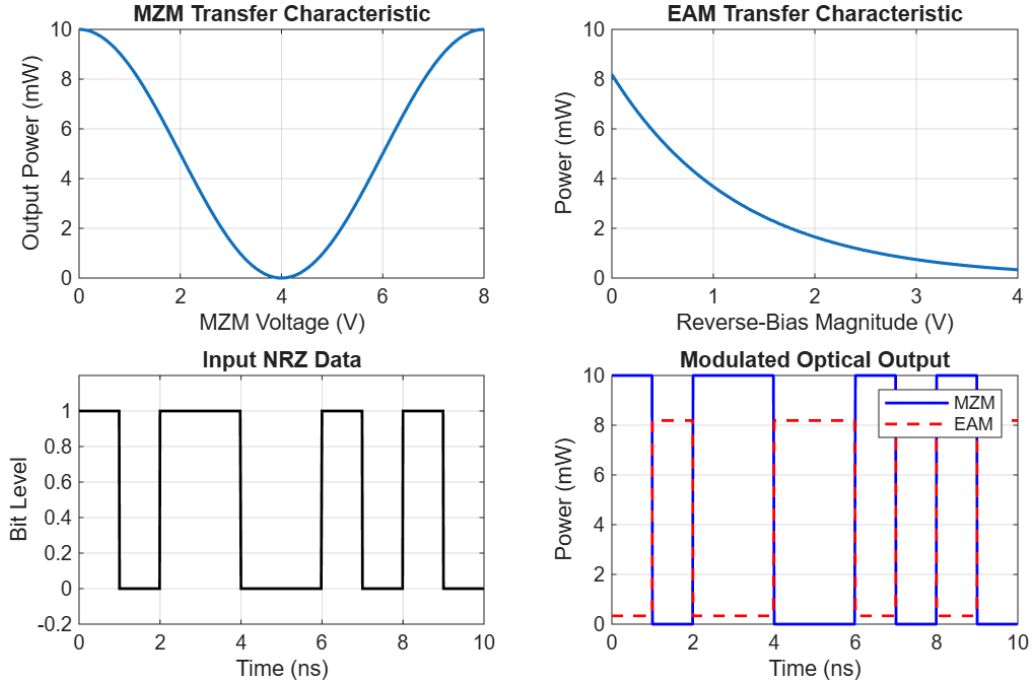


Figure 2: MZM Transfer Characteristic, EAM Transfer Characteristic, Input NRZ Data, and Modulated Optical Output.

2.6 Parameter Comparisons and Model Limitations

Parameter Changes: Adjusting MZM drive voltage amplitudes alters the modulation depth and resulting extinction ratio. Increasing the EAM active length (L) enhances absorption, decreasing output power levels.

Limitations: This is an idealized static comparison; high-frequency parasitic bandwidth limits, optical insertion losses, semiconductor chirp, and fabrication variations are excluded.

2.7 Complete MATLAB Code

```
clear;
clc;
close all;

%% =====
% MZM AND EAM OPTICAL MODULATORS
% Static Transfer Characteristics and NRZ Modulation
% =====

%% Basic Parameters

Pin = 10e-3; % Input optical power (W)

Vpi = 4; % MZM half-wave voltage (V)

%% =====
% PART 1: STATIC MZM CHARACTERISTIC
% =====

V = linspace(0,2*Vpi,1000);

% MZM output power
PMZM = Pin * cos(pi*V/(2*Vpi)).^2;

%% =====
% PART 2: STATIC EAM CHARACTERISTIC
% =====

VR = linspace(0,4,1000);

% EAM parameters
L = 200e-6; % Device length (m)

alpha0 = 1000; % Base absorption coefficient (1/m)

kAlpha = 4000; % Voltage-dependent coefficient

% EAM output power
PEAM = Pin * exp(-(alpha0 + kAlpha*VR)*L);

%% =====
% PART 3: UNIPOLAR NRZ INPUT
% =====

bits = [1 0 1 1 0 0 1 0 1 0];

Rb = 1e9; % Bit rate = 1 Gb/s

sps = 100; % Samples per bit

% NRZ waveform
data = kron(bits,ones(1,sps));
```

```

%% Time axis

Tb = 1/Rb;

dt = Tb/sps;

t = (0:length(data)-1)*dt;

%% =====
% PART 4: MZM OPTICAL MODULATION
% =====

% Map digital data to MZM voltage
%
% Bit 1 -> 0 V
% Bit 0 -> Vpi

VMZM = Vpi*(1-data);

% MZM output optical power
outMZM = Pin * ...
    cos(pi*VMZM/(2*Vpi)).^2;

%% =====
% PART 5: EAM OPTICAL MODULATION
% =====

% Map digital data to reverse bias
%
% Bit 0 -> 0 V
% Bit 1 -> 4 V

VEAM = 4*data;

% EAM output optical power
outEAM = Pin * ...
    exp(-(alpha0 + kAlpha*VEAM)*L);

%% =====
% PART 6: EXTINCTION RATIO
% =====

% MZM maximum and minimum output powers
PmaxMZM = max(outMZM);
PminMZM = min(outMZM);

% EAM maximum and minimum output powers
PmaxEAM = max(outEAM);
PminEAM = min(outEAM);

% Extinction ratio
ERm = 10*log10(PmaxMZM/PminMZM);

```

```

ERe = 10*log10(PmaxEAM/PminEAM);

%% =====
% PART 7: PLOTS
% =====

figure('Color','w');

%% MZM transfer characteristic

subplot(2,2,1);

plot(V,PMZM*1e3,...
     'LineWidth',1.5);

xlabel('MZM Voltage (V)');
ylabel('Output Power (mW)');

title('MZM Transfer Characteristic');

grid on;

%% EAM transfer characteristic

subplot(2,2,2);

plot(VR,PEAM*1e3,...
     'LineWidth',1.5);

xlabel('Reverse-Bias Magnitude (V)');
ylabel('Power (mW)');

title('EAM Transfer Characteristic');

grid on;

%% Input NRZ data

subplot(2,2,3);

stairs(t*1e9,data,...
      'k','LineWidth',1.3);

xlabel('Time (ns)');
ylabel('Bit Level');

title('Input NRZ Data');

ylim([-0.2 1.2]);

grid on;

%% Modulated optical output

```

```

subplot(2,2,4);

plot(t*1e9,...
     outMZM*1e3,...
     'b','LineWidth',1.3);

hold on;

plot(t*1e9,...
     outEAM*1e3,...
     'r--','LineWidth',1.3);

xlabel('Time (ns)');
ylabel('Power (mW)');

legend('MZM','EAM');

title('Modulated Optical Output');

grid on;

%% =====
% DISPLAY RESULTS
% =====

fprintf('\n');
fprintf('===== \n');
fprintf('MZM AND EAM MODULATOR RESULTS \n');
fprintf('===== \n');

fprintf('Input optical power = %.2f mW \n',...
        Pin*1e3);

fprintf('MZM Vpi          = %.2f V \n',...
        Vpi);

fprintf('EAM device length  = %.2f um \n',...
        L*1e6);

fprintf('\n');

fprintf('MZM maximum power    = %.4f mW \n',...
        PmaxMZM*1e3);

fprintf('MZM minimum power     = %.4e mW \n',...
        PminMZM*1e3);

fprintf('MZM extinction ratio = %.2f dB \n',...
        ERm);

fprintf('\n');

fprintf('EAM maximum power     = %.4f mW \n',...
        PmaxEAM*1e3);

fprintf('EAM minimum power     = %.4e mW \n',...

```

```
    PminEAM*1e3);  
  
fprintf('EAM extinction ratio = %.2f dB\n',...  
    ERe);  
  
fprintf('\n');  
  
fprintf('Simulation completed successfully.\n');
```

3 Experiment : Noise, SNR, Q-Factor, and BER of PIN and APD Receivers

3.1 Objectives

- Calculate photodiode responsivity and signal photocurrent.
- Analyze receiver shot noise and thermal noise variance.
- Compare PIN and Avalanche Photodiode (APD) receiver sensitivity, Q-factor, and Bit Error Rate (BER).

3.2 Theory and Relevant Equations

Photodiode responsivity (R) is given by:

$$R = \eta \frac{q\lambda}{hc} \quad (9)$$

For a PIN photodiode, photocurrent is $I_p = RP$. Thermal noise variance introduced by load resistance R_L at absolute temperature T within bandwidth B is:

$$\sigma_T^2 = \frac{4k_BTB}{R_L} \quad (10)$$

PIN noise variances for bit 0 and bit 1 are:

$$\sigma_{0,PIN}^2 = 2qI_dB + \sigma_T^2, \quad \sigma_{1,PIN}^2 = 2q(RP_1 + I_d)B + \sigma_T^2 \quad (11)$$

For an APD with internal multiplication gain M and ionization ratio k_A , the excess-noise factor $F(M)$ is:

$$F(M) = k_AM + (1 - k_A) \left(2 - \frac{1}{M} \right) \quad (12)$$

The APD noise variances and OOK link performance metrics are modeled as:

$$\sigma_{0,APD}^2 = 2qM^2F(M)I_dB + \sigma_T^2, \quad \sigma_{1,APD}^2 = 2qM^2F(M)(RP_1 + I_d)B + \sigma_T^2 \quad (13)$$

$$Q = \frac{\mu_1 - \mu_0}{\sigma_1 + \sigma_0}, \quad BER = \frac{1}{2} \operatorname{erfc} \left(\frac{Q}{\sqrt{2}} \right) \quad (14)$$

3.3 Simulation Parameters

Parameter	Baseline value
Wavelength (λ)	1550 nm
Quantum efficiency (η)	0.80
Noise-equivalent bandwidth (B)	1 GHz
Absolute temperature (T)	300 K
Load resistance (R_L)	1000 Ω
Dark current (I_d)	5 nA
Bit-1 optical power sweep ($P_{1,dBm}$)	−50 to −15 dBm (step 0.2 dB)
APD multiplication gain (M) / Ionization ratio (k_A)	10 / 0.30

3.4 Procedure

1. Define physical constants (q, h, c, k_B) and optical receiver parameters $(\lambda, \eta, B, T, R_L, Id)$.
2. Calculate detector responsivity R and thermal noise variance σ_T^2 .
3. Compute APD excess noise factor $F(M)$ using gain $M = 10$ and ionization ratio $k_A = 0.30$.
4. Sweep received bit-1 optical power from -50 dBm to -15 dBm and evaluate signal current, shot noise, and thermal noise for both PIN and APD receivers.
5. Calculate bit-1 SNR, decision Q-factor, and Bit Error Rate (BER), then plot performance curves to evaluate receiver sensitivity at 10^{-9} BER.

3.5 Results and Generated Figures

Physical Interpretation: Higher received optical power increases signal photocurrent and Q-factor while suppressing BER. Internal APD multiplication boosts signal levels above thermal noise in low-power regimes, yielding superior sensitivity (e.g., -28.2 dBm vs -21.4 dBm at 10^{-9} BER), though excessive multiplication gain increases excess noise.

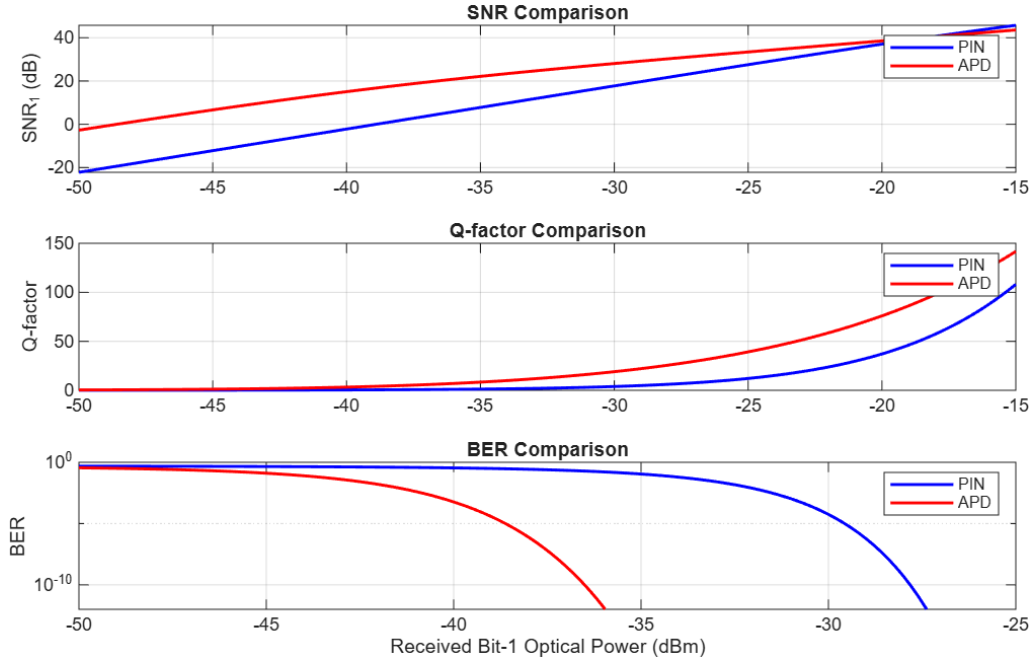


Figure 3: SNR, Q-factor, and BER Performance Comparison Curves for PIN and APD Receivers.

3.6 Parameter Comparisons and Model Limitations

Parameter Changes: Simulating with $M = 1, 5, 10$, and 20 highlights the optimal balance between avalanche gain enhancement and excess noise penalty.

Limitations: The model assumes all dark current is multiplied bulk dark current and employs equal-tail Gaussian threshold approximations.

3.7 Complete MATLAB Code

```

clear;
clc;
close all;

%% =====
% OPTICAL COMMUNICATIONS LABORATORY
% PIN and APD Receivers
% Noise, SNR, Q-factor and BER
% =====

%% Fundamental Constants

q = 1.602e-19;          % Electron charge (C)
h = 6.626e-34;          % Planck constant (J.s)
c = 3e8;                % Speed of light (m/s)
kB = 1.381e-23;         % Boltzmann constant (J/K)

%% =====
% RECEIVER PARAMETERS
% =====

lambda = 1550e-9;       % Wavelength (m)

eta = 0.80;              % Quantum efficiency

% Photodetector responsivity
R = eta*q*lambda/(h*c);

B = 1e9;                 % Noise-equivalent bandwidth (Hz)

T = 300;                 % Temperature (K)

RL = 1000;               % Load resistance (Ohm)

Id = 5e-9;               % Dark current (A)

%% =====
% RECEIVED OPTICAL POWER
% =====

% Optical power corresponding to OOK bit 1
P1dBm = -50:0.2:-15;

% Convert dBm to Watt
P1 = 1e-3 * 10.^(P1dBm/10);

%% =====
% THERMAL NOISE VARIANCE
% =====

thermalVar = ...
    4*kB*T*B/RL;

%% =====

```

```

% PIN RECEIVER
% =====

% Signal current for bit 1
signalPIN = R * P1;

% Noise standard deviation for bit 0
sigma0PIN = sqrt( ...
    2*q*Id*B + thermalVar);

% Noise standard deviation for bit 1
sigma1PIN = sqrt( ...
    2*q*(R*P1 + Id)*B + thermalVar);

% Q-factor
QPIN = ...
    signalPIN ./ (sigma1PIN + sigma0PIN);

% BER
BERPIN = ...
    0.5 * erfc(QPIN/sqrt(2));

%% =====
% APD RECEIVER
% =====

M = 10;                % APD multiplication gain

kA = 0.30;              % Ionization coefficient ratio

% APD excess noise factor
F = kA*M + (1-kA)*(2-1/M);

% APD signal current
signalAPD = M * R * P1;

% Noise for bit 0
sigma0APD = sqrt( ...
    2*q*M^2*F*Id*B + thermalVar);

% Noise for bit 1
sigma1APD = sqrt( ...
    2*q*M^2*F*(R*P1 + Id)*B + thermalVar);

% Q-factor
QAPD = ...
    signalAPD ./ (sigma1APD + sigma0APD);

% BER
BERAPD = ...
    0.5 * erfc(QAPD/sqrt(2));

```

```

%% =====
% SNR CALCULATION
% =====

% Defined as:
% bit-1 signal-current squared / bit-1 noise variance

SNRPIN = ...
    signalPIN.^2 ./ sigma1PIN.^2;

SNRAPD = ...
    signalAPD.^2 ./ sigma1APD.^2;

%% =====
% PLOT RESULTS
% =====

figure('Color','w');

%% SNR

subplot(3,1,1);

plot(P1dBm,...
    10*log10(SNRPIN),...
    'b','LineWidth',1.4);

hold on;

plot(P1dBm,...
    10*log10(SNRAPD),...
    'r','LineWidth',1.4);

ylabel('SNR_1 (dB)');

legend('PIN','APD');

title('SNR Comparison');

grid on;

%% Q-factor

subplot(3,1,2);

plot(P1dBm,...
    QPIN,...
    'b','LineWidth',1.4);

hold on;

plot(P1dBm,...
    QAPD,...
    'r','LineWidth',1.4);

```

```

ylabel('Q-factor');

legend('PIN','APD');

title('Q-factor Comparison');

grid on;

%% BER

subplot(3,1,3);

semilogy(P1dBm,...
    max(BERPIN,1e-20),...
    'b','LineWidth',1.4);

hold on;

semilogy(P1dBm,...
    max(BERAPD,1e-20),...
    'r','LineWidth',1.4);

xlabel('Received Bit-1 Optical Power (dBm)');

ylabel('BER');

legend('PIN','APD');

title('BER Comparison');

ylim([1e-12 1]);

grid on;

%% =====
% DISPLAY BASIC PARAMETERS
% =====

fprintf('\n');
fprintf('===== \n');
fprintf('PIN AND APD RECEIVER RESULTS \n');
fprintf('===== \n');

fprintf('Wavelength          = %.0f nm \n',...
    lambda*1e9);

fprintf('Quantum efficiency      = %.2f \n',...
    eta);

fprintf('Responsivity            = %.3f A/W \n',...
    R);

fprintf('Bandwidth                = %.2f GHz \n',...
    B/1e9);

```

```

fprintf('Temperature          = %.0f K\n',...
        T);

fprintf('Load resistance      = %.0f Ohm\n',...
        RL);

fprintf('Dark current        = %.2e A\n',...
        Id);

fprintf('\n');

fprintf('APD multiplication gain = %.1f\n',...
        M);

fprintf('APD excess-noise factor = %.2f\n',...
        F);

%% =====
% SENSITIVITY AT BER = 1e-9
% =====

targetBER = 1e-9;

idxPIN = find(BERPIN <= targetBER,1);

idxAPD = find(BERAPD <= targetBER,1);

fprintf('\n');
fprintf('===== \n');
fprintf('RECEIVER SENSITIVITY \n');
fprintf('===== \n');

if ~isempty(idxPIN)

    fprintf('PIN sensitivity at BER 1e-9 = %.1f dBm\n',...
            P1dBm(idxPIN));

else

    fprintf('PIN does not reach BER 1e-9 in the selected power range.\n
            ');

end

if ~isempty(idxAPD)

    fprintf('APD sensitivity at BER 1e-9 = %.1f dBm\n',...
            P1dBm(idxAPD));

else

    fprintf('APD does not reach BER 1e-9 in the selected power range.\n
            ');

end

fprintf('\nSimulation completed successfully.\n');

```

4 Experiment : Power Budget and Rise-Time Budget of a Fiber-Optic Link

4.1 Objectives

- Calculate total passive link loss and received optical power.
- Determine whether a fiber link successfully satisfies its power budget margin.
- Estimate system rise time and maximum NRZ bit rate capability from the rise-time budget.

4.2 Theory and Relevant Equations

Total passive link loss accounts for fiber attenuation, connector insertion losses, and splice losses:

$$L_{total} = \alpha L + N_c L_c + N_s L_s \quad (15)$$

Received optical power is $P_r = P_t - L_{total}$. The power budget requires that received power exceeds receiver sensitivity plus a design margin:

$$P_r \geq P_{sens} + M_{design} \quad (16)$$

The aggregate system rise time (t_{sys}) combines transmitter, receiver, and dispersion rise times via root-sum-square addition:

$$t_{sys} = \sqrt{t_{tx}^2 + t_{rx}^2 + t_{chrom}^2 + t_{modal}^2} \quad (17)$$

where chromatic dispersion rise time for a Gaussian source is $t_{chrom} \approx 2.563|D|L\sigma_\lambda$. The standard NRZ transmission bandwidth criterion is:

$$t_{sys} \leq \frac{0.7}{R_b} \quad (18)$$

4.3 Simulation Parameters

Parameter	Baseline value
Transmitter output power (P_{tx})	0 dBm
Receiver sensitivity (P_{sens})	−24 dBm
Design margin (M_{design})	3 dB
Fiber attenuation coefficient (α)	0.25 dB/km
Link distance (L)	50 km
Connectors (N_c, L_c) / Splices (N_s, L_s)	2 (0.5 dB) / 10 (0.1 dB)
Transmitter / Receiver rise times	120 ps / 150 ps
Chromatic dispersion (D) / Width (σ_λ)	17 ps/(nm · km) / 0.10 nm
Transmission bit rate (R_b)	2.5 Gb/s

4.4 Procedure

1. Set up baseline link parameters including transmitter power, receiver sensitivity, and component counts.
2. Compute total passive link loss and received optical power at 50 km, evaluating the remaining power margin.

3. Calculate chromatic dispersion rise time (t_{chrom}) and aggregate system rise time (t_{sys}).
4. Compare system rise time with the allowable limit ($0.7/R_b$) and evaluate link feasibility.

4.5 Results and Generated Figures

Physical Interpretation: The link passes the power budget with a positive margin (6.5 dB remaining), but fails the rise-time budget because system rise time (≈ 290.41 ps) exceeds the allowable limit (280 ps) at 2.5 Gb/s. This proves that optical power sufficiency does not guarantee high-speed operation if dispersion limits bandwidth.

4.6 Parameter Comparisons and Model Limitations

Parameter Changes: Reducing the transmitter RMS spectral width (σ_λ) by half cuts chromatic dispersion rise time, allowing the link to pass the rise-time budget. Increasing transmitter power extends the power-limited maximum link length.

Limitations: Fiber nonlinearities and polarization mode dispersion are excluded.

4.7 Complete MATLAB Code

```
clear;
clc;
close all;

%% =====
%% OPTICAL COMMUNICATION LINK BUDGET
%% Power Budget + Rise-Time Budget
%% =====

%% PART 1: POWER BUDGET

% Transmitter power
Ptx = 0; % dBm

% Receiver sensitivity
Psens = -24; % dBm

% Required design margin
designMargin = 3; % dB

% Fiber attenuation
alpha = 0.25; % dB/km

% Fiber length
L = 50; % km

% Connector parameters
Nc = 2; % Number of connectors
Lc = 0.5; % Loss per connector (dB)

% Splice parameters
Ns = 10; % Number of splices
Ls = 0.1; % Loss per splice (dB)

%% Calculate Fixed Loss
```



```

fixedLoss = Nc*Lc + Ns*Lv;

%% Calculate Total Fiber Loss

fiberLoss = alpha * L;

%% Calculate Total Passive Loss

totalLoss = fiberLoss + fixedLoss;

%% Calculate Received Power

Prx = Ptx - totalLoss;

%% Calculate Remaining Design Margin

remainingMargin = ...
    Prx - Psens - designMargin;

%% Maximum Power-Limited Fiber Length

LmaxPower = ...
    (Ptx - Psens - designMargin - fixedLoss) / alpha;

%% =====
% PART 2: RISE-TIME BUDGET
% =====

% Transmitter rise time
ttx = 120; % ps

% Receiver rise time
trx = 150; % ps

% Chromatic dispersion coefficient
D = 17; % ps/(nm.km)

% RMS spectral width
sigmaLambda = 0.10; % nm

% Modal rise time
% For single-mode fiber, modal dispersion is approximately zero
tmodal = 0; % ps

%% Chromatic Dispersion Rise Time

tchrom = ...
    2.563 * abs(D) * L * sigmaLambda;

%% Total System Rise Time

```

```

tsys = sqrt( ...
    ttx^2 + ...
    trx^2 + ...
    tchrom^2 + ...
    tmodal^2 );

%% =====
% PART 3: ALLOWED RISE TIME
% =====

% Target bit rate
Rb = 2.5e9;                % bit/s

% NRZ rise-time criterion
allowedRise = ...
    (0.7/Rb) * 1e12;

%% Maximum NRZ Bit Rate

RbMax = ...
    0.7 / (tsys*1e-12);

%% =====
% DISPLAY RESULTS
% =====

fprintf('\n');
fprintf('===== \n');
fprintf('OPTICAL COMMUNICATION LINK BUDGET RESULTS \n');
fprintf('===== \n');

fprintf('\n--- POWER BUDGET --- \n');

fprintf('Transmitter power           = %.2f dBm \n', Ptx);
fprintf('Fiber loss                     = %.2f dB \n', fiberLoss);
fprintf('Connector loss                 = %.2f dB \n', Nc*Lc);
fprintf('Splice loss                    = %.2f dB \n', Ns*Lv);
fprintf('Total passive loss             = %.2f dB \n', totalLoss);
fprintf('Received power                 = %.2f dBm \n', Prx);
fprintf('Receiver sensitivity           = %.2f dBm \n', Psens);
fprintf('Remaining design margin        = %.2f dB \n', remainingMargin);
fprintf('Power-limited length           = %.2f km \n', LmaxPower);

fprintf('\n--- RISE-TIME BUDGET --- \n');

fprintf('Transmitter rise time          = %.2f ps \n', ttx);
fprintf('Receiver rise time             = %.2f ps \n', trx);
fprintf('Chromatic dispersion time      = %.2f ps \n', tchrom);
fprintf('Modal rise time                = %.2f ps \n', tmodal);
fprintf('System rise time               = %.2f ps \n', tsys);

fprintf('\n');
fprintf('Target bit rate                = %.2f Gb/s \n', Rb/1e9);

```

```

fprintf('Allowed rise time           = %.2f ps\n', allowedRise);
fprintf('Maximum NRZ bit rate        = %.2f Gb/s\n', RbMax/1e9);

%% =====
% LINK FEASIBILITY CHECK
% =====

fprintf('\n--- LINK CHECK ---\n');

if remainingMargin >= 0
    fprintf('Power budget: PASS\n');
else
    fprintf('Power budget: FAIL\n');
end

if tsys <= allowedRise
    fprintf('Rise-time budget: PASS\n');
else
    fprintf('Rise-time budget: FAIL\n');
end

fprintf('\nSimulation completed successfully.\n');

```

5 Experiment : Line Coding and Eye-Pattern Analysis

5.1 Objectives

- Generate unipolar NRZ, 50% duty-cycle RZ, and Manchester line-coded waveforms.
- Observe the effects of receiver bandwidth limitations and additive noise.
- Interpret digital signal quality and intersymbol interference using eye diagrams.

5.2 Theory and Relevant Equations

Line coding conventions determine pulse shapes: unipolar NRZ remains high for the entire bit duration, RZ returns to zero at mid-bit, and Manchester guarantees a mid-bit transition for clock synchronization. A receiver-equivalent low-pass channel is modeled via a first-order IIR filter:

$$y[n] = a \cdot y[n-1] + (1-a) \cdot x[n] \quad (19)$$

where filter coefficient a depends on cutoff frequency f_c and sampling frequency F_s :

$$a = e^{-2\pi f_c / F_s} \quad (20)$$

An eye diagram is generated by overlaying successive waveform segments over 2-bit windows.

5.3 Simulation Parameters

Parameter	Baseline value
Random seed	'rng(10)'
Number of simulated bits (N_{bits})	500
Samples per bit (sps)	64
Bit rate (R_b)	1 Gb/s
Good / Poor channel cutoffs (f_c)	$0.8R_b$ / $0.2R_b$
Receiver noise standard deviations	0.025 (Good) and 0.070 (Poor)

5.4 Procedure

1. Generate a pseudo-random bit stream of 500 bits using 'rng(10)'.
2. Construct unipolar NRZ, 50% RZ, and Manchester coded time waveforms.
3. Filter the NRZ waveform through low-pass IIR filters representing good ($0.8R_b$) and poor ($0.2R_b$) channel bandwidths.
4. Add zero-mean Gaussian receiver noise to the filtered signals.
5. Overlay aligned 2-bit windows to construct eye patterns and analyze signal degradation.

5.5 Results and Generated Figures

Physical Interpretation: NRZ signals lack guaranteed transitions during consecutive identical bits, whereas Manchester coding ensures frequent transitions for clock recovery at the cost of higher bandwidth. Restricted channel bandwidth slows transition edges, causing intersymbol interference (ISI) that closes the eye horizontally, while noise reduces the vertical eye opening.

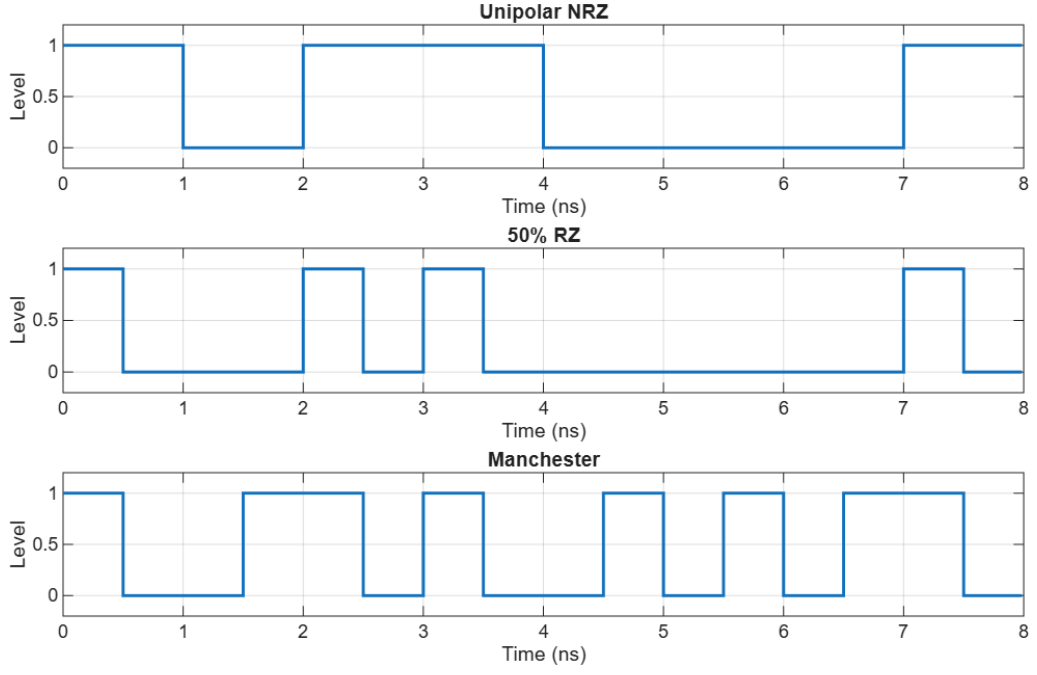


Figure 4: Unipolar NRZ, 50% RZ, and Manchester Time Waveforms.

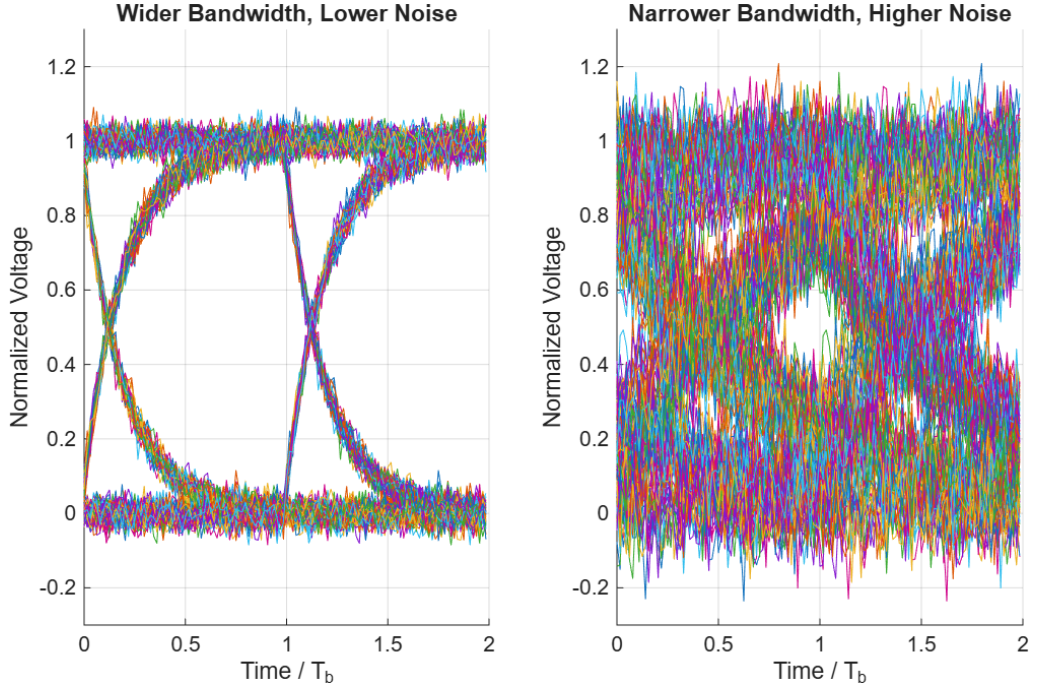


Figure 5: Eye Diagrams for Wider Bandwidth/Lower Noise and Narrower Bandwidth/Higher Noise Channels.

5.6 Parameter Comparisons and Model Limitations

Parameter Changes: Evaluating eye diagrams across cutoff ratios $f_c/R_b = 0.2, 0.5$, and 0.8 shows how bandwidth constraints affect timing jitter and sampling margins.

Limitations: The channel is modeled as a simple first-order low-pass filter without advanced equalization.

5.7 Complete MATLAB Code

```
clear;
clc;
close all;
rng(10);

%% =====
% DIGITAL OPTICAL COMMUNICATION
% Line Coding + Bandwidth Effect + Eye Diagram
% =====

%% Basic Parameters

Nbits = 500;
sps = 64;                % Samples per bit (must be even)
Rb = 1e9;                % Bit rate = 1 Gbps
Fs = Rb * sps;           % Sampling frequency

%% Generate Random Bits

bits = randi([0 1], 1, Nbits);

%% =====
% PART 1: LINE CODING
% =====

% Unipolar NRZ
nrz = kron(bits, ones(1, sps));

% 50% Return-to-Zero
rz = kron(bits, ...
    [ones(1, sps/2), zeros(1, sps/2)]);

% Manchester coding
manchester = ...
    kron(bits, [ones(1, sps/2), zeros(1, sps/2)]) + ...
    kron(1-bits, [zeros(1, sps/2), ones(1, sps/2)]);

%% Show first 8 bits

nshow = 8 * sps;

tshow = (0:nshow-1) / Fs * 1e9;

waveforms = [nrz; rz; manchester];

names = {'Unipolar NRZ', '50% RZ', 'Manchester'};

%% Plot Line-Coded Waveforms

figure('Color', 'w');

for k = 1:3

    subplot(3,1,k);

    stairs(tshow, ...
```

```

        waveforms(k,1:nshow), ...
        'LineWidth',1.4);

xlabel('Time (ns)');
ylabel('Level');

title(names{k});

ylim([-0.2 1.2]);

grid on;

end

%% =====
% PART 2: ELECTRICAL CHANNEL BANDWIDTH
% =====

% Good bandwidth
fcGood = 0.8 * Rb;

% Poor bandwidth
fcPoor = 0.2 * Rb;

% First-order low-pass filter coefficients

aGood = exp(-2*pi*fcGood/Fs);

aPoor = exp(-2*pi*fcPoor/Fs);

%% Filter NRZ signal

yGood = filter( ...
    1-aGood, ...
    [1 -aGood], ...
    nrz);

yPoor = filter( ...
    1-aPoor, ...
    [1 -aPoor], ...
    nrz);

%% =====
% PART 3: ADD RECEIVER NOISE
% =====

% Good channel: lower noise
yGood = yGood + ...
    0.025 * randn(size(yGood));

% Poor channel: higher noise
yPoor = yPoor + ...
    0.070 * randn(size(yPoor));

%% =====

```

```

% PART 4: EYE DIAGRAM
% =====

% Two-bit time window
eyeTime = (0:2*sps-1) / sps;

figure('Color','w');

%% Good bandwidth eye

subplot(1,2,1);

hold on;

for b = 20:200

    idx = (b-1)*sps + (1:2*sps);

    plot(eyeTime, ...
        yGood(idx), ...
        'LineWidth',0.5);

end

xlabel('Time / T_b');
ylabel('Normalized Voltage');

title('Wider Bandwidth, Lower Noise');

ylim([-0.3 1.3]);

grid on;

%% Poor bandwidth eye

subplot(1,2,2);

hold on;

for b = 20:200

    idx = (b-1)*sps + (1:2*sps);

    plot(eyeTime, ...
        yPoor(idx), ...
        'LineWidth',0.5);

end

xlabel('Time / T_b');
ylabel('Normalized Voltage');

title('Narrower Bandwidth, Higher Noise');

ylim([-0.3 1.3]);

```



```

grid on;

%% =====
% PART 5: DISPLAY PARAMETERS
% =====

fprintf('\n');
fprintf('===== \n');
fprintf('DIGITAL OPTICAL COMMUNICATION SIMULATION \n');
fprintf('===== \n');

fprintf('Number of bits      = %d \n', Nbits);
fprintf('Bit rate              = %.2f Gbps \n', Rb/1e9);
fprintf('Samples per bit         = %d \n', sps);
fprintf('Sampling frequency     = %.2f GHz \n', Fs/1e9);

fprintf('\n');
fprintf('Good channel BW        = %.2f GHz \n', fcGood/1e9);
fprintf('Poor channel BW        = %.2f GHz \n', fcPoor/1e9);

fprintf('\nSimulation completed successfully. \n');

```

6 Experiment : WDM Spectrum, Fiber Attenuation, and OTDR Simulation

6.1 Objectives

- Generate a Wavelength Division Multiplexing (WDM) spectrum.
- Observe wavelength-dependent attenuation.
- Estimate fiber attenuation and splice loss from an OTDR trace.
- Add another splice with a one-way loss of 0.5 dB at 25 km.
- Reduce WDM channel spacing and observe spectral overlap.

6.2 Theory and Relevant Equations

WDM allows multiple optical wavelength channels to travel through the same fiber. The received power of channel k is:

$$P_{rx,k} = P_{tx,k} 10^{-\alpha_k L/10}$$

An Optical Time-Domain Reflectometer (OTDR) launches optical pulses and measures re-turning backscatter and reflections to characterize fiber loss and locate events. Distance is calculated from round-trip delay:

$$z = \frac{ct}{2n_g}$$

where n_g is the group refractive index.

This experiment displays the OTDR signal as raw returned-power level in dB:

$$P_{return,dB}(z) = C - 2\alpha z - 2L_{event}(z)$$

Consequently, half the magnitude of the trace slope gives the one-way attenuation. A commercial OTDR's calibrated loss display may use a different convention.

6.3 Simulation Parameters

Parameter	Baseline value
Wavelength grid (λ)	1548 to 1555 nm; 5000 samples
Channel centres (λ_k)	[1549, 1550.5, 1552, 1553.5] nm
Spectral width (σ_λ)	0.06 nm
Channel powers ($P_{channel}$)	1×10^{-3} W per channel
Attenuation coefficients (α_{WDM})	[0.20, 0.21, 0.22, 0.23] dB/km
Fiber length (L)	40 km
Speed of light (c)	3×10^8 m/s
Group refractive index (n_g)	1.468
Distance vector (z)	0 to 40 km (step 0.01 km)
OTDR attenuation (α)	0.20 dB/km (one-way)
First splice position / Loss	15 km; 0.8 dB (one-way)
Second splice position / Loss	25 km; 0.5 dB (one-way)

6.4 Procedure

1. Initialize Wavelength grid from 1548 to 1555 nm with 5000 samples and define 4 channel center wavelengths.
2. Compute unit-area Gaussian spectrum profiles for each channel and accumulate transmitted (Stx) and received (Srx) spectral power densities over a 40 km fiber link.
3. Calculate individual received channel powers in dBm and display results.
4. Simulate Part B by generating a simplified raw OTDR distance and round-trip time trace with a 15 km non-reflective splice and added Gaussian measurement noise.
5. Perform linear regression (polyfit) on selected distance regions before and after the splice event to estimate fiber attenuation and splice loss.
6. Execute Part 3 and Part 4 to evaluate the impact of reduced WDM channel spacing (0.12 nm) and multiple splice events (0.5 dB at 25 km), extracting multi-event losses via specialized fitting intervals.

6.5 Results and Generated Figures

Physical Interpretation: The four WDM channels appear as separate spectral peaks. Because the channels have different attenuation coefficients, their received powers are approximately -8 , -8.4 , -8.8 , and -9.2 dBm. A downward step appears at 15 km in the OTDR trace because of the splice. A step of approximately 1.6 dB in this raw round-trip trace corresponds to a one-way splice loss of approximately 0.8 dB. The estimated attenuation should be approximately 0.20 dB/km.

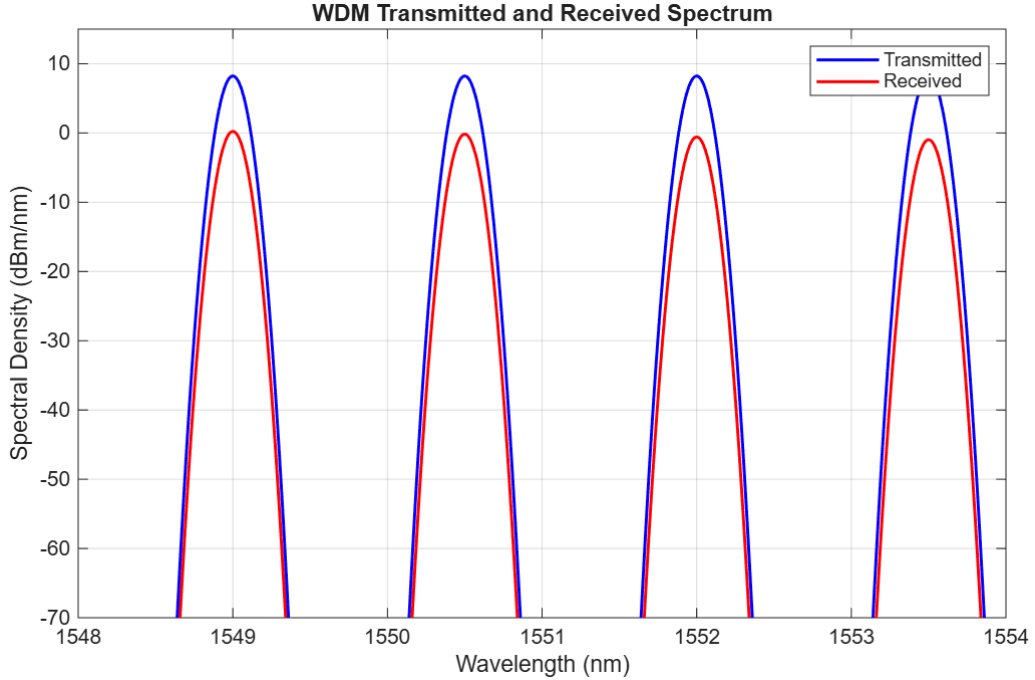


Figure 6: Four-Channel WDM Transmitted and Received Spectrum.

6.6 Parameter Comparisons and Model Limitations

Parameter Changes: Reducing WDM channel spacing causes spectral overlapping, while adding a second splice introduces an additional return-level step on the OTDR trace.

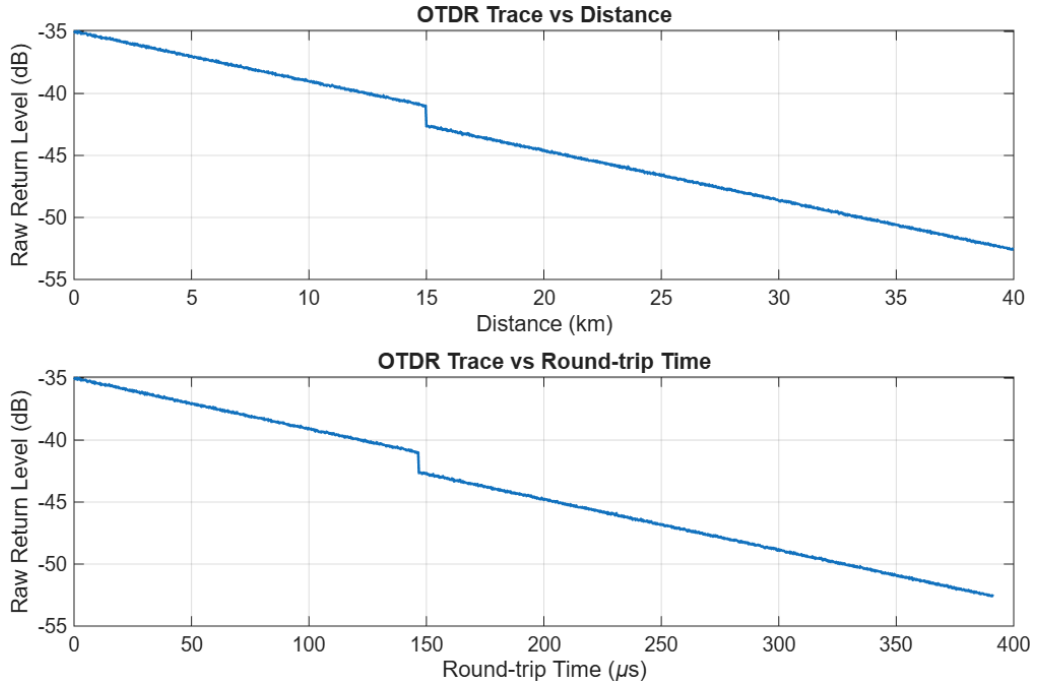


Figure 7: OTDR Trace vs Distance and OTDR Trace vs Round-trip Time.

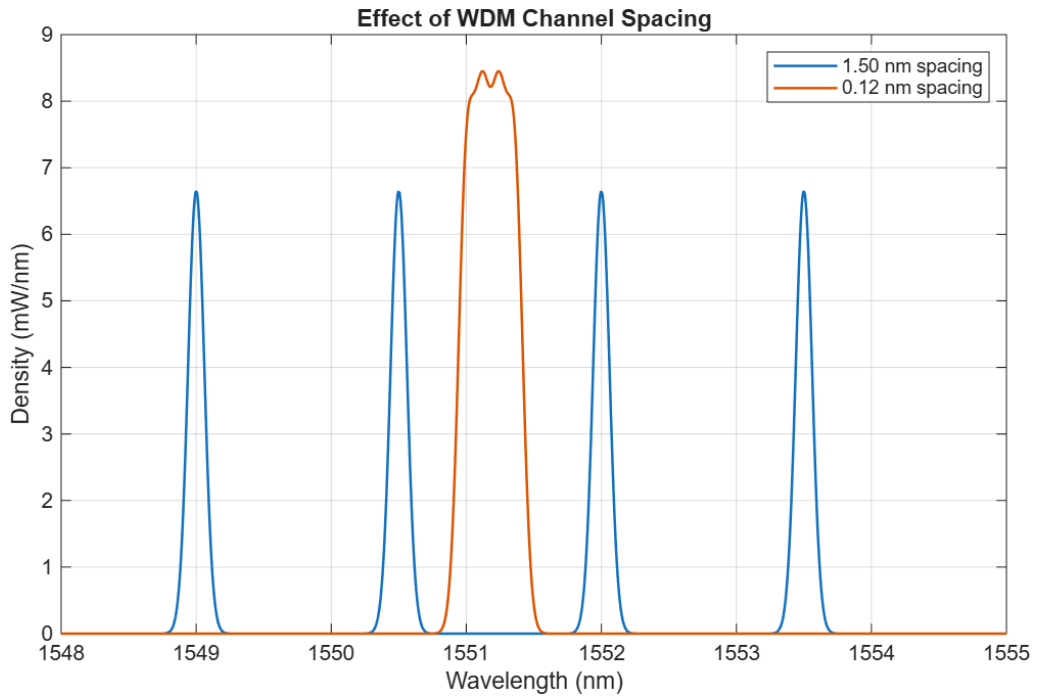


Figure 8: Effect of WDM Channel Spacing (1.50 nm vs 0.12 nm).

Limitations: The OTDR model assumes uniform backscatter and excludes reflective peaks, pulse-width resolution, dead zones, and the instrument noise floor. The WDM section models spectra and attenuation rather than complete modulated-data transmission.

6.7 Complete MATLAB Code

```
clear;
clc;
```

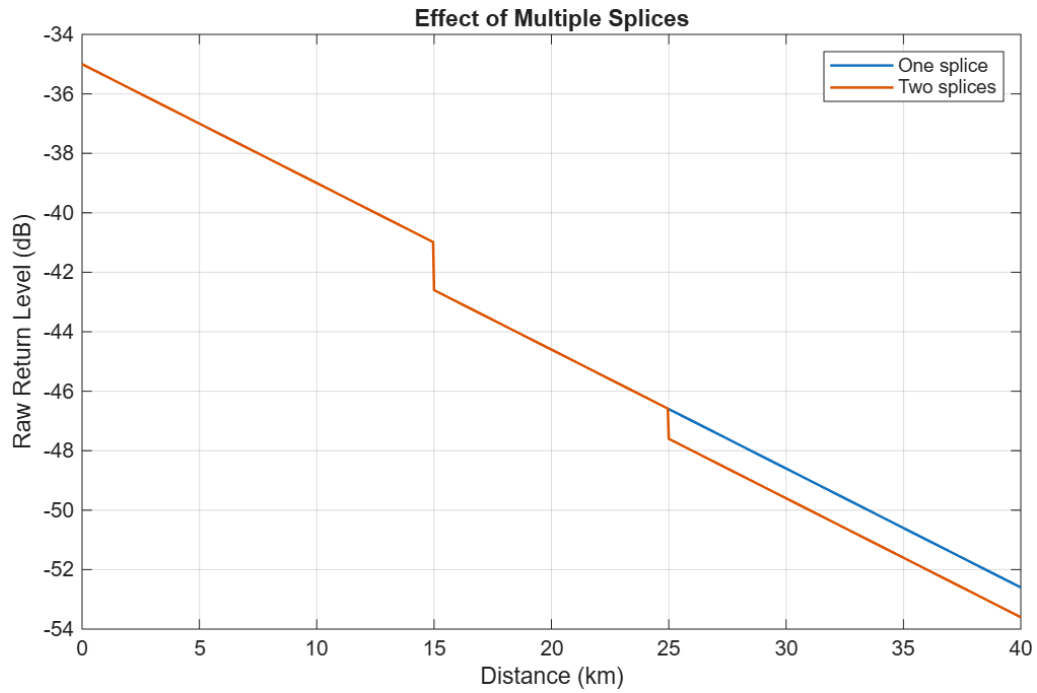


Figure 9: Effect of Multiple Splices on the OTDR Trace.

```
close all;

rng(20);

%% =====
% PART 1: WDM TRANSMISSION
% =====

lambda = linspace(1548,1555,5000);

% Four WDM channel center wavelengths (nm)
centers = [1549 1550.5 1552 1553.5];

% Spectral width (nm)
sigmaLambda = 0.06;

% Channel powers (1 mW each)
Pchannel = 1e-3 * ones(1,4);

% Channel attenuation (dB/km)
alphaWDM = [0.20 0.21 0.22 0.23];

% Fiber length (km)
L = 40;

% Initialize transmitted and received spectra
Stx = zeros(size(lambda));
Srx = zeros(size(lambda));

% Generate WDM channels
for k = 1:length(centers)
    shape = exp(-0.5*((lambda-centers(k))/sigmaLambda).^2) ...
            /(sqrt(2*pi)*sigmaLambda);
```

```

        Stx = Stx + Pchannel(k)*shape;

        % Received spectrum after fiber attenuation
        Srx = Srx + ...
            Pchannel(k)*10^(-alphaWDM(k)*L/10)*shape;
    end

    % Received channel powers
    PrxChannels = Pchannel .* 10.^(-alphaWDM*L/10);

    %% Plot transmitted and received WDM spectra

    figure('Color','w');

    plot(lambda,...
        10*log10(max(Stx,1e-15)/1e-3),...
        'b','LineWidth',1.4);

    hold on;

    plot(lambda,...
        10*log10(max(Srx,1e-15)/1e-3),...
        'r','LineWidth',1.4);

    xlabel('Wavelength (nm)');
    ylabel('Spectral Density (dBm/nm)');
    legend('Transmitted','Received');
    title('WDM Transmitted and Received Spectrum');
    ylim([-70 15]);
    grid on;

    %% Display received channel powers

    disp(' ');
    disp('Received Channel Powers:');

    for k = 1:4
        fprintf('Channel %d (%.1f nm): %.4f dBm\n',...
            k,centers(k),...
            10*log10(PrxChannels(k)/1e-3));
    end

    %% =====
    % PART 2: OTDR ATTENUATION AND SPLICE LOSS
    % =====

    c = 3e8;                % Speed of light (m/s)
    ng = 1.468;             % Group refractive index

    % Distance (km)
    z = 0:0.01:40;

    % Fiber attenuation
    alpha = 0.20;           % dB/km

    % First splice

```

```

splicePosition = 15; % km
spliceLoss = 0.8; % dB

% Generate ideal OTDR trace
traceDB = -35 ...
    - 2*alpha*z ...
    - 2*spliceLoss*(z >= splicePosition);

% Add measurement noise
traceDB = traceDB + 0.04*randn(size(z));

%% Calculate round-trip time

roundTripTime = 2*ng*(z*1000)/c;

%% Select regions for attenuation fitting

mask1 = z >= 2 & z <= 12;
mask2 = z >= 18 & z <= 30;

% Linear fitting before and after splice
fit1 = polyfit(z(mask1),traceDB(mask1),1);
fit2 = polyfit(z(mask2),traceDB(mask2),1);

% Estimate attenuation
alphaEstimated = -fit1(1)/2;

% Estimate splice loss
spliceEstimated = ...
    (polyval(fit1,15)-polyval(fit2,15))/2;

%% Plot OTDR trace

figure('Color','w');

subplot(2,1,1);

plot(z,traceDB,'LineWidth',1.2);

grid on;

xlabel('Distance (km)');
ylabel('Raw Return Level (dB)');

title('OTDR Trace vs Distance');

subplot(2,1,2);

plot(roundTripTime*1e6,traceDB,'LineWidth',1.2);

grid on;

xlabel('Round-trip Time (\mus)');
ylabel('Raw Return Level (dB)');

title('OTDR Trace vs Round-trip Time');

```

```

%% Display estimated values

fprintf('\n');
fprintf('=====\n');
fprintf('OTDR RESULTS\n');
fprintf('=====\n');

fprintf('Attenuation = %.6f dB/km\n',...
    alphaEstimated);

fprintf('Splice loss = %.6f dB\n',...
    spliceEstimated);

%% =====
% PART 3: EFFECT OF WDM CHANNEL SPACING
% =====

% Close channel spacing = 0.12 nm
centersClose = 1551 + (0:3)*0.12;

StxClose = zeros(size(lambda));

for k = 1:4
    shape = exp(-0.5*((lambda-centersClose(k))/sigmaLambda).^2) ...
        /(sqrt(2*pi)*sigmaLambda);

    StxClose = StxClose + Pchannel(k)*shape;
end

%% Plot channel spacing comparison

figure('Color','w');

plot(lambda,Stx*1e3,...
    'LineWidth',1.2);

hold on;

plot(lambda,StxClose*1e3,...
    'LineWidth',1.2);

xlabel('Wavelength (nm)');
ylabel('Density (mW/nm)');

legend('1.50 nm spacing',...
    '0.12 nm spacing');

title('Effect of WDM Channel Spacing');

grid on;

%% =====
% PART 4: EFFECT OF TWO SPLICES

```



```

% =====

% First splice at 15 km
baseClean = -35 ...
    - 2*alpha*z ...
    - 2*spliceLoss*(z >= 15);

% Second splice:
% Position = 25 km
% One-way splice loss = 0.5 dB

twoClean = baseClean ...
    - 2*0.5*(z >= 25);

%% Plot one-splice vs two-splice OTDR trace

figure('Color','w');

plot(z,baseClean,...
    'LineWidth',1.2);

hold on;

plot(z,twoClean,...
    'LineWidth',1.2);

grid on;

xlabel('Distance (km)');
ylabel('Raw Return Level (dB)');

legend('One splice','Two splices');

title('Effect of Multiple Splices');

%% =====
% PART 5: ESTIMATE BOTH SPLICE LOSSES
% =====

rng(20);

% Add noise
twoNoisy = twoClean + 0.04*randn(size(z));

% Fitting regions
a = z >= 2 & z <= 12;
b = z >= 18 & z <= 23;
d = z >= 28 & z <= 36;

% Linear fits
fa = polyfit(z(a),twoNoisy(a),1);
fb = polyfit(z(b),twoNoisy(b),1);
fd = polyfit(z(d),twoNoisy(d),1);

% First splice loss at 15 km
loss15 = ...

```

```

        (polyval(fa,15)-polyval(fb,15))/2;

% Second splice loss at 25 km
loss25 = ...
        (polyval(fb,25)-polyval(fd,25))/2;

%% Display final results

fprintf('\n');
fprintf('=====\n');
fprintf('MULTIPLE SPLICE RESULTS\n');
fprintf('=====\n');

fprintf('First splice   (15 km) = %.6f dB\n',...
        loss15);

fprintf('Second splice (25 km) = %.6f dB\n',...
        loss25);

fprintf('\n');
fprintf('Expected attenuation = %.2f dB/km\n',...
        alpha);

fprintf('Expected first splice = %.2f dB\n',...
        spliceLoss);

fprintf('Expected second splice = %.2f dB\n',...
        0.5);

fprintf('\nSimulation completed successfully.\n');

```